

MindMaze

Adaptive Reasoning & Puzzle Assessment System

BUILD YOUR OWN PROJECT (BYOP)

Submitted By

Gauri Chaudhary

B.Tech – Computer Science and Engineering
(Artificial Intelligence & Machine Learning)

VIT Bhopal University

Build Your Own Project (BYOP)

Academic Year 2026

Introduction

MindMaze is a Java-based command-line application designed to provide an interactive reasoning and puzzle assessment experience. The system allows users to solve challenges from different reasoning categories and difficulty levels.

Unlike a basic quiz application, MindMaze uses a rule-based adaptive mechanism to adjust the challenge level according to the user's performance. The system also provides score calculation, accuracy analysis, level unlocking, category-wise performance analysis, and assessment history.

The project demonstrates the practical use of Java programming, object-oriented design, file-based data storage, input validation, and modular software architecture.

Problem Statement

Traditional quiz and reasoning applications generally provide questions at fixed difficulty levels and mainly focus on displaying a final score. Such systems may not provide an adaptive assessment experience based on the user's performance.

MindMaze addresses this limitation by providing a rule-based adaptive reasoning assessment system. The difficulty level can change according to the user's performance, while the system also tracks accuracy, score, category-wise performance, and assessment history.

The project aims to create a simple, interactive, and modular command-line system for practicing and assessing reasoning skills.

Objectives

- To develop a Java-based command-line reasoning assessment system.
- To provide multiple reasoning categories.
- To implement Beginner, Intermediate, and Advanced difficulty levels.
- To implement rule-based adaptive difficulty.
- To provide performance-based level unlocking.
- To calculate score and accuracy.
- To provide category-wise performance analysis.
- To maintain assessment history.
- To implement input validation and error handling.
- To demonstrate modular and object-oriented Java programming.

Functional Requirements

User Setup

- The system should allow the user to enter their name.
- The system should create a personalized assessment session.

Category Selection

- The system should provide multiple reasoning categories:
 - Pattern Recognition
 - Logical Reasoning
 - Numerical Reasoning
 - Mixed Challenge

Difficulty Selection

- The system should provide three difficulty levels:
 - Beginner
 - Intermediate
 - Advanced Assessment
- The system should display questions according to the selected category and difficulty.
- The system should accept and validate user answers.
- The system should calculate the score and accuracy.

Adaptive Difficulty

- The system should adjust the difficulty according to the user's performance.
- A minimum accuracy of 60% should be required to unlock the next level.

Performance Analysis

- The system should display score and accuracy.
- The system should provide category-wise performance information.
- The system should maintain assessment history.

Non Functional Requirements

Usability

- The system should provide a simple and easy-to-use command-line interface.
- Instructions and menu options should be clearly displayed.

Performance

- The system should process user input and display results efficiently.
- Question evaluation and score calculation should be performed without unnecessary delay.

Reliability

- The system should handle valid and invalid inputs appropriately.
- Assessment data should be stored correctly.

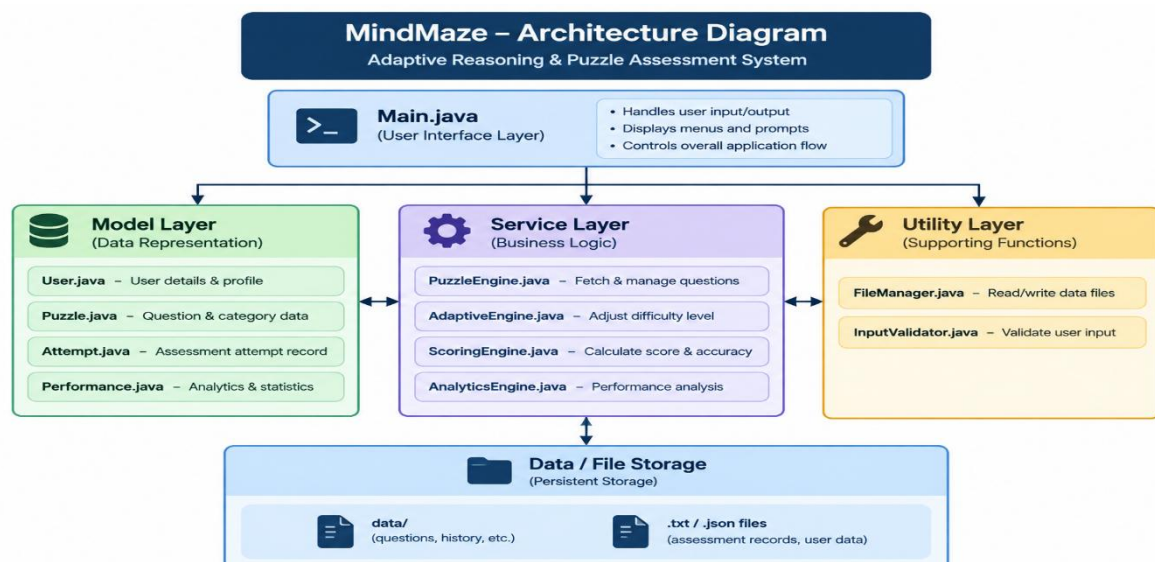
Maintainability

- The application should use modular Java classes.
- Different responsibilities should be separated into appropriate packages and services.

Security and Data Integrity

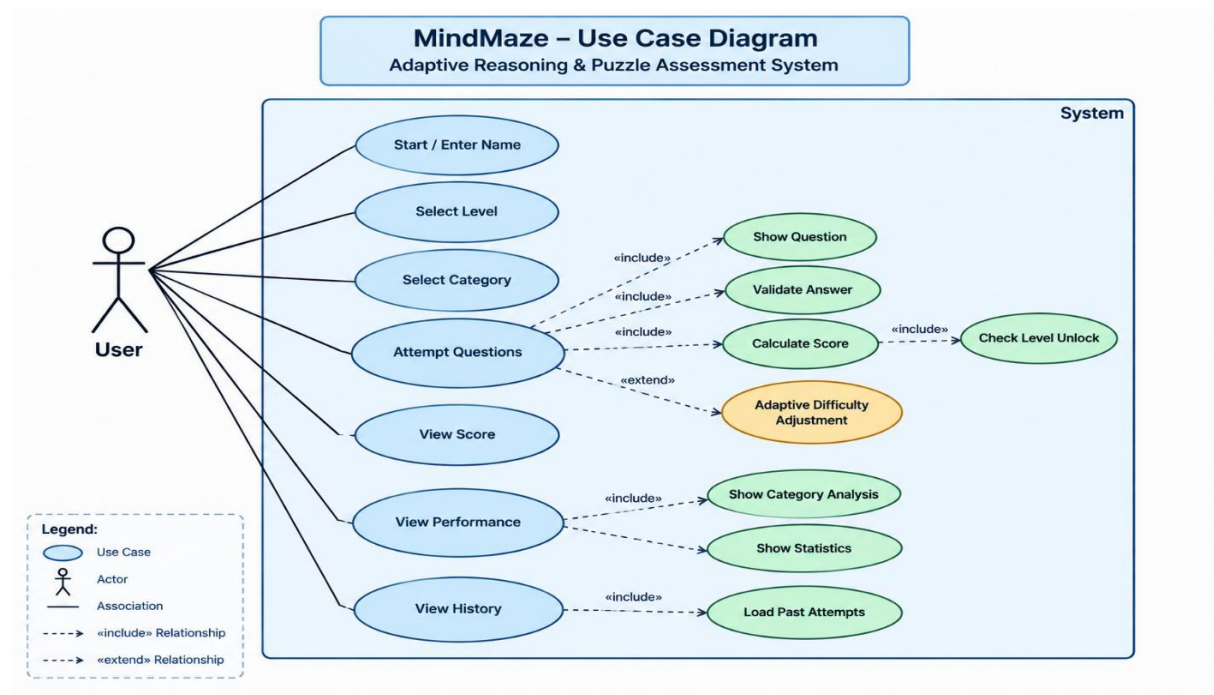
- User assessment information should be stored consistently.
- Invalid input should not cause the application to terminate unexpectedly.

System Architecture



The architecture of MindMaze follows a modular design. The Main class provides the command-line interface and coordinates the application flow. The Model layer represents entities such as users, puzzles, attempts, and performance data. The Service layer handles puzzle processing, adaptive difficulty, scoring, and analytics. Utility classes support input validation and file management. Assessment information is stored using file-based storage.

Use Case Diagram



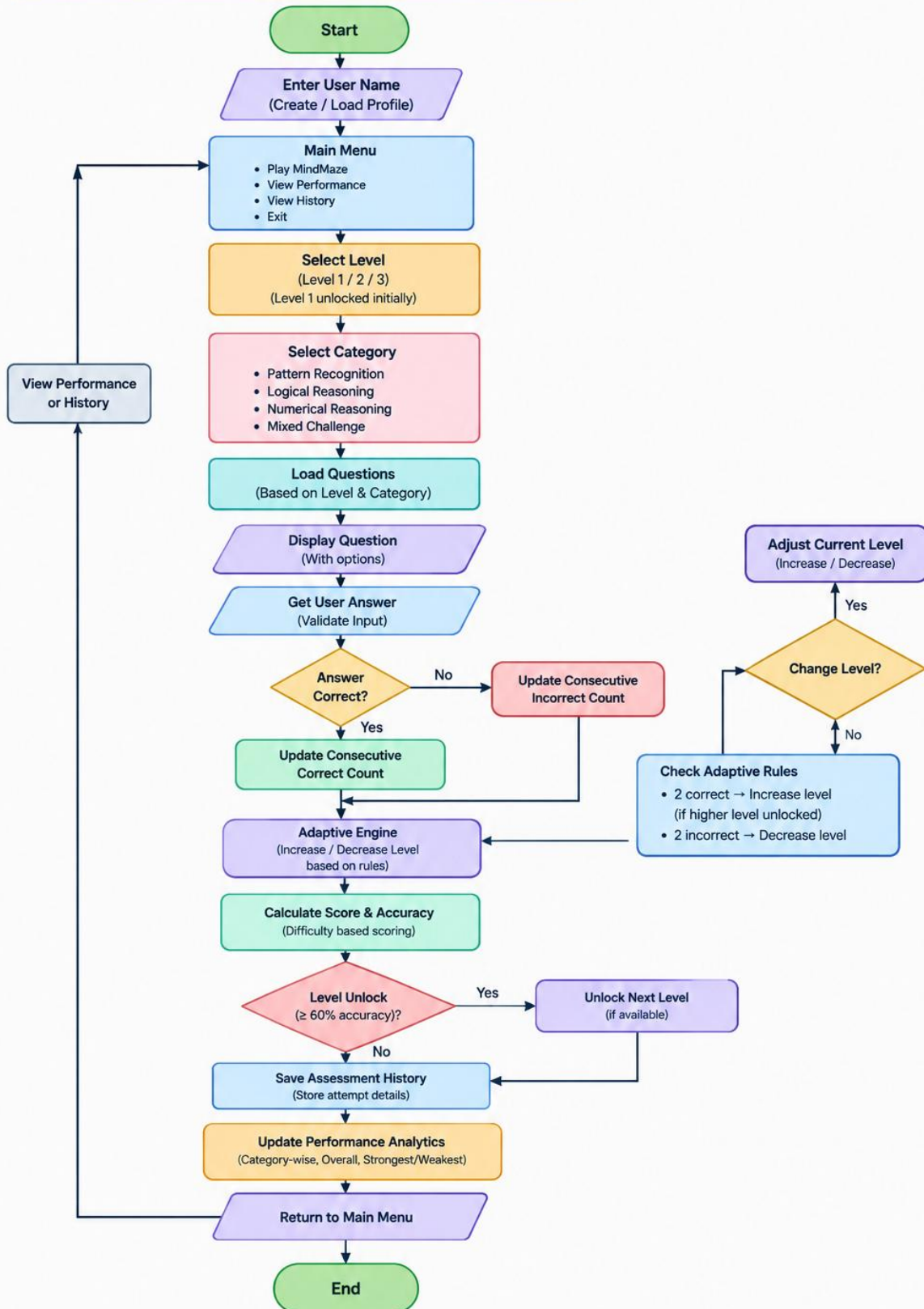
The use case diagram represents the interaction between the user and the MindMaze system. The user can enter their name, select a difficulty level and puzzle category, attempt challenges, answer questions, view scores and performance reports, check assessment history, and exit the application.

Workflow Diagram

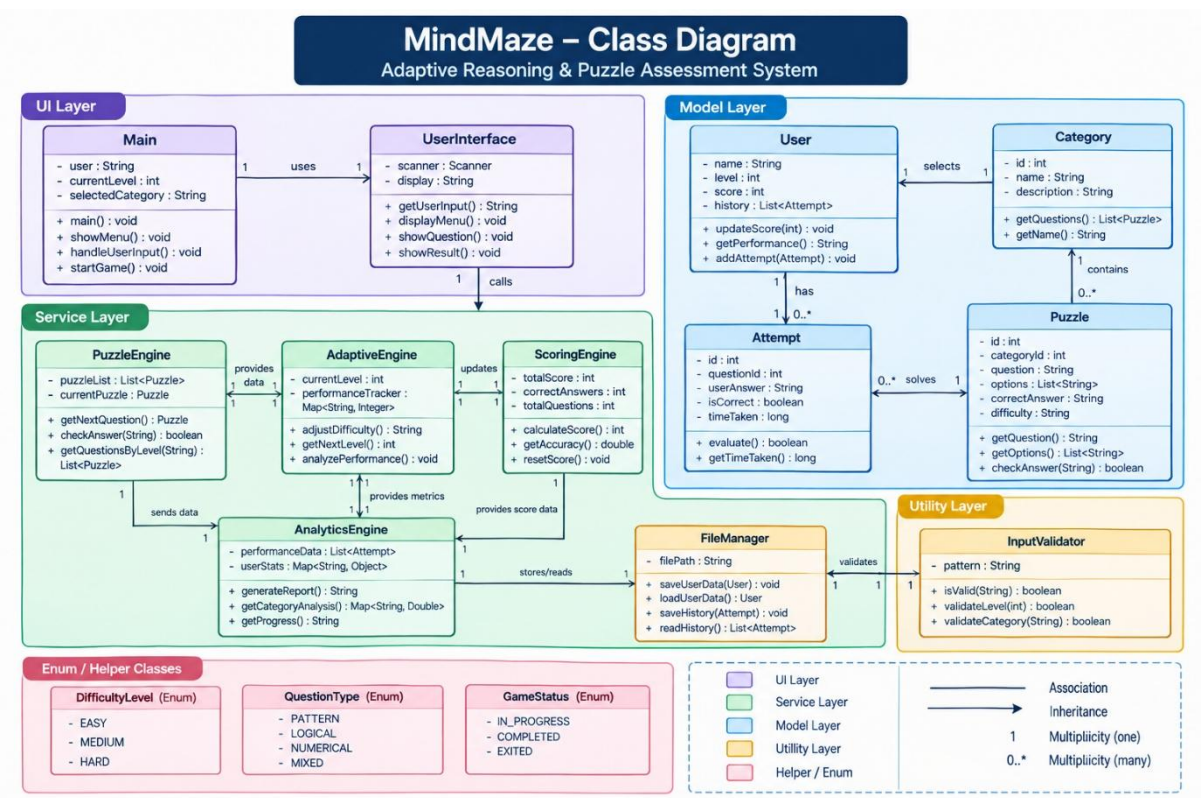
The workflow of MindMaze starts with entering the user's name and selecting the required difficulty level and puzzle category. The system then presents questions to the user and evaluates each response. Based on the user's performance, the adaptive mechanism determines the next challenge level. The system calculates the score and accuracy, checks level unlocking conditions, saves assessment history, and displays the final performance information.

MindMaze – Workflow Diagram

Adaptive Reasoning & Puzzle Assessment System



Class Diagram



The class diagram represents the main classes and their relationships in the MindMaze system. The model classes manage user, puzzle, attempt, and performance-related data. Service classes handle puzzle processing, adaptive difficulty, scoring, and analytics. Utility classes provide supporting functions such as input validation and file management. These classes work together to implement the overall assessment workflow.

Implementation Details

MindMaze is implemented using Java and follows a modular object-oriented structure. The application is divided into different components so that user interaction, puzzle management, scoring, adaptive difficulty, data handling, and validation can be managed separately.

Technologies Used

- Programming Language: Java
- JDK: JDK 26
- IDE: IntelliJ IDEA
- Version Control: Git and GitHub
- Interface: Command Line Interface (CLI)
- Data Storage: File-based storage

Main Modules

- Model: Manages entities such as users, puzzles, attempts, and performance.
- Service: Handles puzzle processing, scoring, adaptive difficulty, and analytics.
- Utility: Provides input validation and file-management functionality.
- Main: Controls the overall application flow and user interaction.

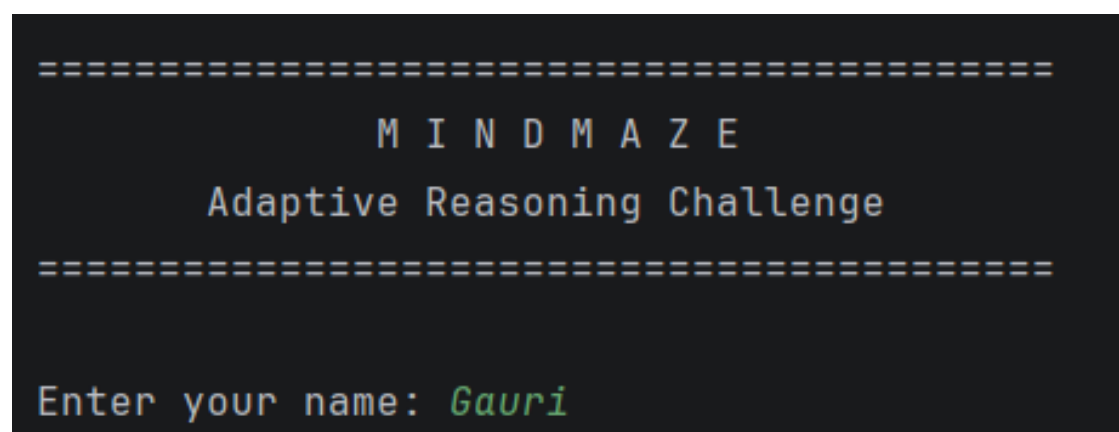
Difficulty Levels and Adaptive Mechanism

MindMaze provides three difficulty levels: Beginner, Intermediate, and Advanced. Each level contains challenges with different levels of reasoning complexity. The system uses a rule-based adaptive mechanism to adjust the challenge level according to the user's performance. A minimum accuracy of 60% is required to unlock the next level.

LEVEL	DIFFICULTY	SCORE PER QUESTION
LEVEL 01	Beginner	10
LEVEL 02	Intermediate	20
LEVEL 03	Advanced	30

Screenshots & Results

Application Start Screen



Main Menu

```
Welcome to MindMaze, Gauri!

===== MAIN MENU =====
Player : Gauri
Unlocked Level : 1
-----
1. Play MindMaze
2. Performance Report
3. Assessment History
4. About MindMaze
5. Exit
=====
Enter your choice: 1
```

Difficulty Level Selection

```
===== LEVEL SELECT =====
1. LEVEL 1 - BEGINNER [UNLOCKED]
2. LEVEL 2 - INTERMEDIATE [LOCKED]
3. LEVEL 3 - ADVANCED [LOCKED]
4. Back to Main Menu
=====
Choose level: 1
```

Assessment and Question Screen

```
===== LEVEL 1 - BEGINNER =====
1. Pattern Recognition
2. Logical Reasoning
3. Numerical Reasoning
4. Mixed Challenge
5. Back
Choose category: |
```

```
=====
                        CHALLENGE STARTED
Level      : LEVEL 1 - BEGINNER
Category   : Pattern Recognition
Questions: 5
=====

Question 1 of 5
-----
Find the next number: 5, 10, 15, 20, ?
-----
1. 22
2. 24
3. 25
4. 30
Your answer:
```

Performance Report

```
=====
                PERFORMANCE SUMMARY
=====
Total Questions : 5
Correct Answers : 1
Accuracy        : 20%
Total Score     : 10
Strongest Area  : Pattern Recognition
Weakest Area    : Logical Reasoning
Adaptive Level  : Beginner
=====

Highest Unlocked Level : 1
```

Assessment History

```
===== ASSESSMENT HISTORY =====
Completed challenges are stored in:
data/history.txt

Challenges completed: 1
=====
```

Main Menu

```
===== ABOUT MINDMAZE =====
MindMaze is an adaptive reasoning
and puzzle assessment system.

Players progress through three levels:
Level 1 → Beginner
Level 2 → Intermediate
Level 3 → Advanced

The system analyses performance and
adjusts the challenge difficulty.
=====
```

Testing

MindMaze was tested to verify the correctness and reliability of its major features. Different inputs and assessment scenarios were used to check the behaviour of the application.

Functional Testing

Test Case	Expected Result
Enter valid user name	User session starts successfully
Select a category	Selected category is loaded
Select difficulty level	Selected level is displayed
Answer a question correctly	Score is updated
Answer a question incorrectly	Incorrect response is recorded
Complete an assessment	Score and accuracy are calculated
Achieve required accuracy	Next level can be unlocked
Enter invalid input	Validation message is displayed
View assessment history	Previous assessment information is displayed

Challenges & Learnings

Challenges Faced

- Designing a rule-based adaptive difficulty mechanism.
- Managing different puzzle categories and difficulty levels.
- Handling invalid user inputs in a CLI environment.
- Maintaining assessment history using file-based storage.
- Organizing the application into modular Java components.
- Calculating score and accuracy correctly.

Key Learnings

- Improved understanding of Java and object-oriented programming.
- Learned to structure a project using separate model, service, and utility components.
- Gained experience with input validation and file handling.
- Learned how adaptive logic can be implemented using performance-based rules.
- Gained practical experience with Git and GitHub for version control.
- Improved understanding of software testing and documentation.

Future Enhancements

MindMaze can be further enhanced with additional features to make the assessment system more comprehensive and interactive.

- **Timed Challenges:** Add time limits for selected challenges.
- **Larger Puzzle Database:** Add more questions and problem variations.
- **More Categories:** Introduce additional reasoning and aptitude categories.
- **Leaderboard:** Add a leaderboard to compare assessment scores.
- **GUI Version:** Develop a graphical user interface for easier interaction.
- **Database Integration:** Replace file-based storage with a database system.
- **Progress Visualization:** Provide graphical representations of user performance and progress.

Conclusion

MindMaze successfully demonstrates the development of a Java-based adaptive reasoning and puzzle assessment system. The application provides multiple reasoning categories, three difficulty levels, performance-based level unlocking, score and accuracy calculation, category-wise analysis, and assessment history.

The project also demonstrates practical implementation of object-oriented programming, modular software design, input validation, file-based data handling, testing, and version control using Git and GitHub.

Overall, the project provided practical experience in designing, implementing, testing, and documenting a complete Java-based application.

References

- Oracle Java Documentation -
<https://docs.oracle.com/en/java/>
- GitHub Documentation - <https://docs.github.com/>
- IntelliJ IDEA Documentation -
<https://www.jetbrains.com/help/idea/>
- Project Repository GitHub -
<https://github.com/gaurichaudhary2/MindMaze/tree/master>
- Project source code, diagrams, test outputs, and documentation developed as part of the MindMaze project.